

Relatório - Tarefa 18: Produto Matriz-Vetor (Distribuição por Colunas)

Introdução

Nesta tarefa, o cálculo do produto matriz-vetor da Tarefa 17 foi reimplementado com foco na distribuição da matriz por **colunas**, em vez de linhas.

Foram desenvolvidas duas versões de distribuição de colunas:

1. **matvec_col_vector**: Utiliza apenas `MPI_Type_vector`.
2. **matvec_col_resized**: Combina `MPI_Type_vector` e `MPI_Type_create_resized` para corrigir os limites (extent) do tipo de dados derivado.

Resolução e Diferenças de Distribuição

Distribuição por Linhas (Tarefa 17)

- **Comunicação**: O vetor x (de tamanho N) inteiro é transmitido por `MPI_Bcast` para todos os processos. A matriz A sofre `MPI_Scatter` em blocos contíguos de memória, o que é extremamente eficiente pois nenhum empacotamento é necessário pelo MPI.
- **Processamento**: Cada processo calcula os seus elementos locais finais do vetor y .
- **Finalização**: Um `MPI_Gather` é feito em blocos contíguos de y para fechar os dados na raiz. Não requer operações aritméticas de rede.

Distribuição por Colunas (Tarefa 18)

- **Comunicação**: Apenas fatias (segmentos de tamanho N/P) de x são distribuídas aos processos. Por outro lado, `MPI_Scatter` distribui blocos espaçados da matriz A correspondentes às colunas alocadas. Como as fatias da matriz A de tamanho $M \times N$ armazenada em array 1D sequencial não contêm todos os dados adjacentes na memória para as colunas, cria-se a necessidade do uso dos tipos de dados derivados para leitura entre *strides*.
- **Processamento**: No código em C para armazenamento por linhas (row-major), as colunas estão intercaladas. O `MPI_Scatter` usando vetor envia as partes isoladas mas chega do outro lado contíguo no buffer receptor. Ao usar apenas `MPI_Type_vector` (Versão 1 da Tarefa 18), o MPI se baseia na verdadeira “largura” percorrida do primeiro byte até o último byte. O salto ou “extent” real do tipo vector compreende o final de todos os saltos no último bloco da matriz. Portanto, quando a raiz envia blocos para processos $P > 0$, o ponteiro que calcula os blocos das demais *ranks* varre muito além da matriz real, resultando em dados incorretos e violação de memória. A segunda versão corrige isso ajustando explicitamente o extent com `MPI_Type_create_resized`. O “extent” (espaço entre os inícios de blocos no Scatter para processos seguidos) é definido para o deslocamento das primeiras colunas ($N_{local} \times \text{sizeof}(\text{double})$).

- **Finalização:** O vetor local de contribuições de cada um possui tamanho inteiro igual a M , pois todos têm contribuições para a soma de todas as linhas baseadas nas suas devidas colunas locais. Utiliza-se um MPI_Reduce local do tamanho completo do vetor para alcançar a soma (MPI_SUM), incorrendo num alto custo de comunicação extra pois exige repasses e somas no meio da rede em relação a todos os de M posições.

Análise de Desempenho e Acesso à Memória

1. Acesso Global à Memória Base da Matriz Pelo aspecto de afinidade de *cache* de CPU: A operação primária $A[i * \text{local_N} + j]$ continua rodando de modo encadeado sobre arrays 1D na CPU tanto na versão row-major de linhas quanto de colunas, visto que iteramos internamente j após a divisão. Por outro lado, do lado do nodo primário que emite e empacota essas informações na rede, as trocas das subestruturas não mapeadas são mais lentas no lado “Scatter” que exige repacks pelo MPI para transmitir *strides* ao invés do despejo *Memcpy-like* da versão em linhas.

2. Tráfego da Rede e Operações

- No caso das *linhas*, temos um Bcast(tamanho N) em uma chamada e um Gather(tamanho M/P).
- No caso das *colunas*, o x reduz o papel (Scatter N/P para cada), todavia as saídas exigem o Reduce de forma vetorial inteira de tamanho M (onde todos interagem em MPI_SUM). Na prática o balanceamento da latência do Reduce com relação ao bloco do Scatter/Process é comumente pior e consome mais recursos de rede/CPU. Na maioria dos clusters, a versão *matvec* por linhas apresenta tempo superior por maximizar as facilidades dos sub-sistemas de memória de modo consecutivo, em vez de depender de processamento de layout disperso via rede nas intersecções de *Reduce*.

3. Validação matvec_col_vector x matvec_col_resized A distribuição vetorial estrita *matvec_col_vector* se portará de modo equivocado devido à discrepância de extent, invariavelmente registrando respostas erradas e terminando em -1.0 ou *segfault*. Conforme evidenciado pela execução em cluster, quando o número de processos é 1 ($P = 1$) a versão funciona sem divisão. No entanto, quando $P > 1$, ocorre o erro *Out of Bounds* de memória no recebimento pelo Rank 2 e aborto da submissão geral. Devido a esse comportamento, as execuções para $P > 1$ com *col_vector* falham.

A *matvec_col_resized*, no entanto, realiza a geometria com mapeamento fiel fornecido pelas redimensionadas da biblioteca OpenMPI, finalizando validada de forma rigorosa em semelhança absoluta na sua completude em relação às soluções de linhas clássicas.

Resultados e Comparação de Desempenho

Abaixo estão os resultados consolidados das execuções no cluster para a distribuição por linhas (row da Tarefa 17) e por colunas usando *resized* (*col_resized*), variando o número de processos (P) e a dimensão da matriz quadrada ($N = M$):

Dimensão (N)	Processos (P)	Tempo Linhas (s)	Tempo Colunas (s)
1200	1	0.0054	0.0058
2400	1	0.0233	0.0246
4800	1	0.0881	0.0958
11520	1	0.5054	0.5351
16000	1	0.9830	1.0380
35360	1	4.8928	5.1648
1200	2	0.0081	0.0085
2400	2	0.0235	0.0353
4800	2	0.0922	0.1289
11520	2	0.5012	0.7271
16000	2	0.9785	1.4385
35360	2	4.8575	6.9747
1200	4	0.0082	0.0092
2400	4	0.0234	0.0293
4800	4	0.0949	0.1512
11520	4	0.5079	0.8327
16000	4	0.9692	1.5506
35360	4	4.7132	7.6779

Discussão dos Resultados

Através da observação dos tempos coletados, podemos constatar claramente que **a distribuição por linhas superou a distribuição por colunas em todas as instâncias de teste.**

Motivos evidenciados:

1. **Sobrecarga (Overhead) de Empacotamento de Tipos Derivados:** Para $P = 1$, vemos que há uma diferença constante (entre 5% e 8%) favorável à versão por linhas. O MPI_Scatter da versão em linhas envia um grande bloco linear e direto de memória, não precisando empacotar estruturas com saltos de stride, enquanto o vector na col_resized causa *overhead* extra de empacotamento em buffers temporários pelo MPI antes mesmo de transitar pela rede.
2. **Custo de Redução vs Coleta Contígua:** Quando passamos para $P = 2$ e $P = 4$, a diferença de tempo aumenta expressivamente, especialmente em dimensões altas. Em $N = 35360$ com $P = 4$, a versão base levou ~ 4.71 segundos, enquanto a versão com colunas demandou ~ 7.67 segundos (uma piora superior a 60%). Isso se deve ao fato de que, no final do algoritmo por colunas, cada processo computa uma

versão completa local do vetor y (de M posições). Para unificar isso, a comunicação requer a operação global de soma `MPI_Reduce(MPI_SUM)` para todos os M elementos entre os nodos. Em contrapartida, a versão por linhas cada processo preenche de forma estanque a sua respectiva fração contígua de y (tamanho M/P) que não possui sobreposições de valores. Desse modo, requer apenas um `MPI_Gather` perfeitamente otimizado que cola os pedaços lado a lado para o nodo principal, ignorando toda operação de ALU na rede e movendo muito menos dados nas ramificações concorrentes.

Apêndice: Códigos-Fonte

Apêndice A: Implementação `matvec_col_vector.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    int rank, size;
    int M, N;
    double *A = NULL, *x = NULL, *y = NULL;
    double *local_A = NULL, *local_x = NULL, *local_y = NULL;
    int local_N;
    double start_time, end_time, local_time, max_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc != 3) {
        if (rank == 0)
            fprintf(stderr, "Uso: %s <M> <N>\n", argv[0]);
        MPI_Finalize();
        return 1;
    }

    M = atoi(argv[1]);
    N = atoi(argv[2]);

    if (N % size != 0) {
        if (rank == 0)
            fprintf(stderr, "Erro: Numero de colunas N (%d) deve ser\n", N, size);
        MPI_Finalize();
        return 1;
    }

    local_N = N / size;
```

```

// Alocacao
local_A = (double *)malloc(M * local_N * sizeof(double));
local_x = (double *)malloc(local_N * sizeof(double));
local_y = (double *)malloc(M * sizeof(double));

if (rank == 0) {
    A = (double *)malloc(M * N * sizeof(double));
    y = (double *)malloc(M * sizeof(double));
    x = (double *)malloc(N * sizeof(double));

    // Inicializacao
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            A[i * N + j] = 1.0;
    for (int i = 0; i < N; i++)
        x[i] = 1.0;
}

MPI_Datatype col_type;
MPI_Type_vector(M, local_N, N, MPI_DOUBLE, &col_type);
MPI_Type_commit(&col_type);

MPI_Barrier(MPI_COMM_WORLD);
start_time = MPI_Wtime();

// Scatter do vetor x (segmentos correspondentes)
MPI_Scatter(x, local_N, MPI_DOUBLE, local_x, local_N, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

// Scatter das colunas da matriz A
// 0 buffer de recebimento local_A armazena os blocos de colunas de
forma contígua!
MPI_Scatter(A, 1, col_type, local_A, M * local_N, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

// Produto matriz-vetor local (contribuicao parcial)
for (int i = 0; i < M; i++) {
    local_y[i] = 0.0;
    for (int j = 0; j < local_N; j++) {
        local_y[i] += local_A[i * local_N + j] * local_x[j];
    }
}

// Reduce das contribuicoes parciais
if (rank == 0) {
    MPI_Reduce(local_y, y, M, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
} else {
    MPI_Reduce(local_y, NULL, M, MPI_DOUBLE, MPI_SUM, 0,

```

```

MPI_COMM_WORLD);
    }

    end_time = MPI_Wtime();
    local_time = end_time - start_time;

    MPI_Reduce(&local_time, &max_time, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        int validation = 1;
        for (int i = 0; i < M; i++) {
            if (y[i] != (double)N) {
                validation = 0;
                break;
            }
        }
        if (validation)
            printf("%f\n", max_time);
        else
            printf("-1.0\n");
        free(A);
        free(y);
        free(x);
    }

    MPI_Type_free(&col_type);
    free(local_A);
    free(local_x);
    free(local_y);
    MPI_Finalize();
    return 0;
}

```

Apêndice B: Implementação matvec_col_resized.c

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    int rank, size;
    int M, N;
    double *A = NULL, *x = NULL, *y = NULL;
    double *local_A = NULL, *local_x = NULL, *local_y = NULL;
    int local_N;
    double start_time, end_time, local_time, max_time;

    MPI_Init(&argc, &argv);

```

```

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (argc != 3) {
    if (rank == 0)
        fprintf(stderr, "Uso: %s <M> <N>\n", argv[0]);
    MPI_Finalize();
    return 1;
}

M = atoi(argv[1]);
N = atoi(argv[2]);

if (N % size != 0) {
    if (rank == 0)
        fprintf(stderr, "Erro: Numero de colunas N (%d) deve ser
multiplo do numero de processos (%d).\n", N, size);
    MPI_Finalize();
    return 1;
}

local_N = N / size;

// Alocacao
local_A = (double *)malloc(M * local_N * sizeof(double));
local_x = (double *)malloc(local_N * sizeof(double));
local_y = (double *)malloc(M * sizeof(double));

if (rank == 0) {
    A = (double *)malloc(M * N * sizeof(double));
    y = (double *)malloc(M * sizeof(double));
    x = (double *)malloc(N * sizeof(double));

    // Inicializacao
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            A[i * N + j] = 1.0;
    for (int i = 0; i < N; i++)
        x[i] = 1.0;
}

MPI_Datatype col_type, resized_col_type;
MPI_Type_vector(M, local_N, N, MPI_DOUBLE, &col_type);
MPI_Type_create_resized(col_type, 0, local_N * sizeof(double),
&resized_col_type);
MPI_Type_commit(&resized_col_type);

MPI_Barrier(MPI_COMM_WORLD);
start_time = MPI_Wtime();

```

```

    // Scatter do vetor x (segmentos correspondentes)
    MPI_Scatter(x, local_N, MPI_DOUBLE, local_x, local_N, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

    // Scatter das colunas da matriz A com o novo extent
    // O buffer de recebimento local_A armazena os blocos de colunas de
forma contígua!
    MPI_Scatter(A, 1, resized_col_type, local_A, M * local_N, MPI_DOUBLE,
0, MPI_COMM_WORLD);

    // Produto matriz-vetor local (contribuicao parcial)
    for (int i = 0; i < M; i++) {
        local_y[i] = 0.0;
        for (int j = 0; j < local_N; j++) {
            local_y[i] += local_A[i * local_N + j] * local_x[j];
        }
    }

    // Reduce das contribuicoes parciais
    if (rank == 0) {
        MPI_Reduce(local_y, y, M, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    } else {
        MPI_Reduce(local_y, NULL, M, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
    }

    end_time = MPI_Wtime();
    local_time = end_time - start_time;

    MPI_Reduce(&local_time, &max_time, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        int validation = 1;
        // A implementacao vector falhara, mas a resized deve fornecer
resultados exatos
        for (int i = 0; i < M; i++) {
            if (y[i] != (double)N) {
                validation = 0;
                break;
            }
        }
        if (validation)
            printf("%f\n", max_time);
        else
            printf("-1.0\n");

        free(A);
    }

```

```

        free(y);
        free(x);
    }

    MPI_Type_free(&col_type);
    MPI_Type_free(&resized_col_type);
    free(local_A);
    free(local_x);
    free(local_y);
    MPI_Finalize();
    return 0;
}

```

Script de Submissão (job.sh)

```

#!/bin/bash
#SBATCH --job-name=matvec_tarefa18
#SBATCH --time=0-0:45
#SBATCH --partition=intel-128
#SBATCH --mem=64G
#SBATCH --ntasks=4
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1

make clean
make

cd ../17-tarefa
make clean
make
cd ../18-tarefa

echo "Versao,Processos,Dimensao,Tempo" > resultados.csv

DIMENS0ES=(1200 2400 4800 11520 16000 35360)
PROCESS0S=(1 2 4)

for p in "${PROCESS0S[@]"; do
    for dim in "${DIMENS0ES[@]"; do

        # Tarefa 17 (Distribuicao por Linhas)
        res_row=$(mpirun -np $p ../17-tarefa/matvec $dim $dim)
        echo "row,$p,$dim,$res_row" >> resultados.csv

        # Tarefa 18 (Distribuicao por Colunas - Vector Only)
        # res_col_vec=$(mpirun -np $p ../matvec_col_vector $dim $dim)
        # echo "col_vector,$p,$dim,$res_col_vec" >> resultados.csv

        # Tarefa 18 (Distribuicao por Colunas - Resized)
    }
}

```

```
res_col_res=$(mpirun -np $p ./matvec_col_resized $dim $dim)
echo "col_resized,$p,$dim,$res_col_res" >> resultados.csv

done
done
```